IST 387: Introduction to Applied Data Science
Lab 1: Part 1 Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- The steps in doing Data Science
- Identifying Data Problems
- Installing R
- Using R
- Creating and using Vectors
- Subsetting Vectors

Preeti Jagadev

The steps in doing Data Science

Data scientists play the most active role in the four A's of data:

1. **Data Architecture:** A data scientist provides inputs on how the data needs to be organized to support the analysis, visualization, and presentation of the data to the appropriate people.
2. **Data Acquisition:** Acquisition focuses on how the data is collected and represented prior to analysis and presentation.
3. **Data Analysis:** The data scientists are most involved at this step. Data analysis is used to summarize the data, use portions of data (samples) to make inferences about larger context, and visualization of the data by presenting it in tables, graphs, and animations. Although there are many technical, mathematical, and statistical aspects involved, the ultimate audience is always people. Thus, there is a need for excellent communication skills in data science.
4. **Data Archiving:** Preservation of data in a form that makes it highly reusable. This is a difficult task because it is so hard to anticipate all the future use of the data.

Identifying Data Problems

- Gather the domain knowledge related to the area of interest from subject matter experts or online sources.
- Figure out anomalies: The unusual things that happen for better or for worse.
- Explore risk and uncertainty involved.

Installing R

- If your computer has the Windows, Mac-OS-X, or a Linux operating system, install R using http://cran.r-project.org/
- If you have difficulty installing this software, then you can refer the book by *Thomas P. Hogan called Bare-Bones R: A brief Introductory guide (2009, Thousand oaks, CA: SAGE).*
- You can also refer the site *readthedocs.org (http://tinyurl.com/872ngtt).*
- If you are having trouble installing R and RStudio, you can try https://rstudio.cloud, which is a web-based version of R studio.

Using R

You will be introduced to command prompt and index

If you are running R and you type a command, for e.g.: 1+1, then it will appear after the *greater than symbol (>),* as shown below:

Preeti Jagadev

(Please note in all the codes, the dark blue color is what we type, and the black color is what R generates as output).

```
> 1+1
```

Here the symbol > is the **command prompt**.

Now, if you press the **enter or return key**, this command will be processed by R, and a result appears on the next line as shown below:

```
> 1+1
[1] 2
```

When the processing is complete, another command prompt appears (>) on the next line, which indicates that R is ready for your next command, as shown below:

```
> 1+1
[1] 2
>
```

Please note, here [1] is the **index,** that helps to keep track of the results that R displays.

Creating and Using Vectors

Consider a list of ages of family members 43, 42, 12, 8, and 5, for Dad, Mom, Daughter, Son, and their dog, respectively. This is a list of integers (integers are numbers that do not have any decimal points). We can create a vector (the most basic form of data storage in R) of integers in R using the **c () command** (where, c stands for combine), as shown below:

```
> c(43,42,12,8,5)
[1] 43 42 12  8  5
```

We observe that R echoes the list back, because we did not ask it to store the vector anywhere. Thus, lets assign these numbers into a storage area **myFamilyAges.** We need to use a left pointing arrow (**assignment arrow**) as shown below:

```
> myFamilyAges <- c(43,42,12,8,5)
```

When we press enter after typing this command, we get an empty command prompt at the output, as shown below:

Preeti Jagadev

```
> myFamilyAges <- c(43,42,12,8,5)
>
```

If you want to know what is in a data object (using R), then just type the name of the object, and R will report it back to you.

```
> sum(myFamilyAges)
[1] 110
```

The command above (in blue text), asks R to add together all the numbers in **myFamilyAges,** which is 110. In the above command, **sum()** is called a **function**. Let's write a command to compute the mean of all the ages, as shown below:

```
> mean(myFamilyAges)
[1] 22
```

In the command above, **mean()** is a **function**. Next, lets compute the highest and the lowest range of all the numbers, as shown below:

```
> range(myFamilyAges)
[1]  5 43
```

In the command above, **range()** is a **function**.

Please note, you can freely try things out in R at any time without fear of breaking anything. If R cannot understand your command, it will calmly respond with an error message and will not make any further changes until you give it a new command.

It is important to remember that **R is case sensitive**. This means that **myFamilyAges is different from myFamilyages.** If we misspell the name of the data object by messing up the capitalization, we will get an error as shown below:

```
> myFamilyAges
[1] 43 42 12  8  5
> myFamilyages
Error: object 'myFamilyages' not found
```

Now, try out the example I have shown below and try to figure out what is happening here:

Preeti Jagadev

```
> myRange <- range(myFamilyAges)
> myRange
[1]  5 43
```

Some good resources for help regarding R as follows:

http://stackoverflow.com

http://www.statmethods.net/

Subsetting Vectors

We can access individual elements within a vector to select a subset of the elements in the vector. For example, we can select the second, third, and the fifth element of the myFamilyAges vector, as shown below:

```
> myFamilyAges[ c(2,3,5) ]
[1] 42 12  5
```

We can also define a sequential range of values (indices) of the vector myFamilAges as shown below:

```
> myFamilyAges[ c(3:5) ]
[1] 12  8  5
```

If the indices above are made negative, then those vector elements are removed and whatever is left over is returned, as shown below:

```
> myFamilyAges[c(-3:-5)]
[1] 43 42
```

We can also tell R, via a list of **TRUE'S** and **FALSE's,** which vector elements to include, by using one of the options given below:

- **Option 1:** A vector of **TRUE** and **FALSE** (once for each element in the vector), as shown below:

```
> myFamilyAges[c(TRUE, FALSE, TRUE, FALSE, FALSE)]
[1] 43 12
```

- **Option 2:** A variable that contains TRUE or FALSE for each element of the vector, as shown below:

```
> selectedFamily <- c(TRUE, FALSE, TRUE, FALSE, FALSE)
> myFamilyAges[ selectedFamily ]
[1] 43 12
```

(Both the options give the same results. I prefer using the option 2).

Next, lets test which family members are elder than 21 years. This is an important concept that needs to be understood well.

```
> myFamilyAges > 21
[1]  TRUE  TRUE FALSE FALSE FALSE
```

To obtain the actual values that are greater than 21, we can write the following command:

```
> selectedFamily <- myFamilyAges > 21
> myFamilyAges[ selectedFamily ]
[1] 43 42
```

In the code above, the expression myFamilyAges >21 applies the conditional test to every element of myFamilyAges and produces a vector of TRUEs and FALSEs that is the same length as myFamilyAges.

Let's write a conditional statement to identify the family members whose age is not 12, as shown below:

```
> myFamilyAges[ myFamilyAges != 12 ]
[1] 43 42  8  5
```

OR

```
> SelectedAge <- myFamilyAges != 12
> SelectedAge
[1]  TRUE  TRUE FALSE  TRUE  TRUE
> myFamilyAges[ SelectedAge ]
[1] 43 42  8  5
```

Here the symbol **!** = means not equal to.

Finally let's construct an expression that performs multiple conditional tests, such as all the family members older than 6 but younger than 20:

```
> MySelectedAges <- myFamilyAges > 6 & myFamilyAges < 20
> myFamilyAges[ MySelectedAges ]
[1] 12  8
```

OR

```
> myFamilyAges > 6 & myFamilyAges < 20
[1] FALSE FALSE  TRUE  TRUE FALSE
> myFamilyAges[ c(FALSE, FALSE, TRUE, TRUE, FALSE) ]
[1] 12  8
```

OR

```
> myFamilyAges[ myFamilyAges > 6 & myFamilyAges < 20 ]
[1] 12  8
```

For all the commands used above, the **R console** (an interface where you can directly type in commands) was used.

To promote quality and efficiency in the R code that we create, we must build something that is reusable. Thus, we must use an integrated development environment (IDE).

If you think of R as a piece of canvas lying on the table, R studio is like an elegant picture frame where you can mount your canvas. R hangs in the middle of R studio. Because we are working with data, we also need ways of inspecting the data, both its structure and its contents. As you probably noticed, it gets tedious doing this at the R console. As an IDE for R, RStudio allows us to control and monitor our code, our data, our output, and our graphics in a way that supports the creation of reusable code sequences.

Some important Functions in R

c ( ): Creates a vector.

min( ): Finds the minimum number in a vector.

mean(): Finds the average for the entire vector.

range(): Finds the min and max values for the vector.

sum(): Finds the total for the entire vector.

length(): Finds the length (number of elements) of the vector.